

StringBuffer Insert and Append

In this chapter you will learn:

1. [How to append to StringBuffer](#)
2. [How to use the chained append\(\) method](#)
3. [How to add data in the middle of a StringBuffer](#)

Append to StringBuffer

The `append()` method appends the string to the end. It is overloaded to have the following forms. We can get usage for each method by the parameters.

`append(boolean b)` is to append boolean value to the `StringBuffer`.

- `StringBuffer append(boolean b)`
- `StringBuffer append(char c)`
- `StringBuffer append(char[] str)`
- `StringBuffer append(char[] str, int offset, int len)`
- `StringBuffer append(CharSequence s)`
- `StringBuffer append(CharSequence s, int start, int end)`
- `StringBuffer append(double d)`
- `StringBuffer append(float f)`
- `StringBuffer append(int i)`
- `StringBuffer append(long lng)`
- `StringBuffer append(Object obj)`
- `StringBuffer append(String str)`
- `StringBuffer append(StringBuffer sb)`
- `StringBuffer appendCodePoint(int codePoint)`

The following code shows how to use the various `append` methods.

```
public class Main {
    public static void main(String[] argv) {
        StringBuffer sb = new StringBuffer();
        sb.append(true); // from java2s.com
        sb.append("java2s.com");
        sb.append(1.2);
        sb.append('a');
        System.out.println(sb.toString());
    }
}
```

The output:

truejava2s.com1.2a

java2s.com

`String.valueOf( )` is called for each parameter to obtain its string representation.

Chained append() method

The buffer itself is returned by each version of `append()`. This allows subsequent calls to be chained together, as shown in the following example:

```
public class Main {
    public static void main(String args[]) {
        String s; /* java2s.com */
        int a = 42;
        StringBuffer sb = new StringBuffer(40);
        s = sb.append("a = ").append(a).append("!").toString();
        System.out.println(s);
    }
}
```

The output of this example is shown here:

a = 42!

java2s.com

Insert to a StringBuffer

The `insert( )` method inserts one string into another. It is overloaded to accept values of all the simple types, plus `Strings`, `Objects`, and `CharSequences`. It calls `String.valueOf( )` to obtain the string representation of the value it is called with.

String

- [Java String type](#)
- [Java String Concatenation](#)
- [Java String Creation](#)
- [Java String Compare](#)
- [Java String Search](#)
- [Java String and char array](#)
- [Java String Conversion](#)
- [String trim, length, is empty, and substring](#)
- [String replace](#)

StringBuffer

- [StringBuffer class](#)
- [StringBuffer Insert and Append](#)
- [StringBuffer length and capacity](#)
- [StringBuffer char operation](#)
- [StringBuffer Operations](#)
- [Search within StringBuffer](#)
- [StringBuffer to String](#)

StringBuilder

- [StringBuilder](#)
- [StringBuilder insert and append](#)
- [StringBuilder length and capacity](#)
- [StringBuilder get, delete, set char](#)
- [StringBuilder delete, reverse](#)
- [StringBuilder search with indexOf and lastIndexOf](#)
- [StringBuilder to String](#)

String Format

- [Formatter class](#)
- [Format Specifier](#)
- [Format String and characters](#)
- [Format integer value](#)
- [Format decimal](#)
- [Scientific notation format](#)
- [Format octal and hexadecimal value](#)
- [Format date and time value](#)
- [Escape Formatter](#)
- [Minimum Field Width](#)
- [Specifying Precision](#)
- [Format Flags](#)
- [Uppercase Option](#)
- [Formatter Argument Index](#)
- [Align left and right](#)
- [Left and right padding a string](#)

String Format Utilities

- [Abbreviate string](#)
- [Capitalize a string](#)
- [Uncapitalize a string](#)
- [Utility class for right padding](#)
- [Left padding](#)
- [Centers a String](#)
- [Transforms words](#)

- `StringBuffer insert(int offset, boolean b)`
- `StringBuffer insert(int offset, char c)`
- `StringBuffer insert(int offset, char[] str)`
- `StringBuffer insert(int index, char[] str, int offset, int len)`
- `StringBuffer insert(int dstOffset, CharSequence s)`
- `StringBuffer insert(int dstOffset, CharSequence s, int start, int end)`
- `StringBuffer insert(int offset, double d)`
- `StringBuffer insert(int offset, float f)`
- `StringBuffer insert(int offset, int i)`
- `StringBuffer insert(int offset, long l)`
- `StringBuffer insert(int offset, Object obj)`
- `StringBuffer insert(int offset, String str)`

The following sample program inserts strings:

```
public class Main {  
    public static void main(String[] argv) {  
        StringBuffer sb = new StringBuffer();  
        sb.append(true); /*from java 2s. com*/  
        sb.append("java2s.com");  
        sb.append(1.2);  
        sb.append('a');  
  
        sb.insert(3, "JAVA@S.COM");  
        System.out.println(sb.toString());  
    }  
}
```

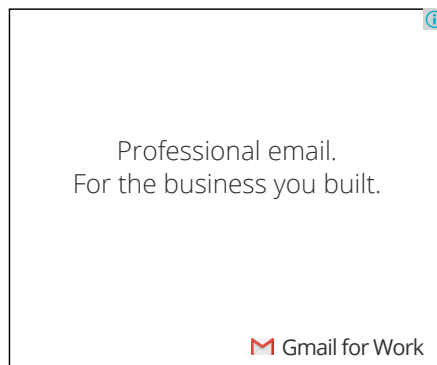
The output:

truJAVA@S.COMejava2s.com1.2a java2s.com

Next chapter...

What you will learn in the next chapter:

1. [What is difference between length and capacity of a Java StringBuffer](#)
2. [How to Pre-allocate space](#)
3. [How to shorten a StringBuffer](#)



[« Previous](#)

[Next »](#)